

APLIKACJE WEBOWE

Projektowanie API REST

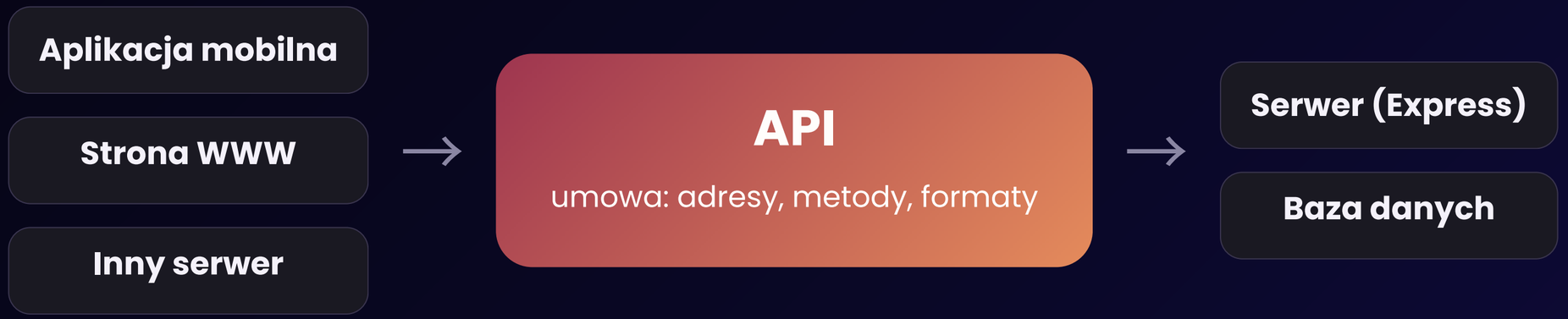
Zasady REST · metody HTTP · idempotentność
projektowanie endpointów · wersjonowanie

L7



by ***Mateusz Węglewski***

Czym jest API?



> *„API to zestaw reguł opisujących, jak jeden program może się łączyć oraz komunikować z innym.”*
>
> OVHcloud, Centrum wiedzy: „Co to jest API REST?”

Słowniczek na dziś

Endpoint
Metoda + adres, np. GET /uczniowie/1: jedno „okienko” API

URI / URL
Adres zasobu. URL to URI z lokalizacją, np. https://...

JSON
Tekstowy format danych: {"imie": "Ala", "klasa": "4TI"}

Nagłówek HTTP
Dodatkowa informacja w żądaniu lub odpowiedzi, np. Content-Type

Query string
Część adresu po znaku „?”, np. ?klasa=4TI

Kod statusu
Trzycyfrowy wynik żądania, np. 200 OK, 404 Not Found

CRUD
Create, Read, Update, Delete: cztery podstawowe operacje na danych

RFC
Dokument IETF opisujący standard internetu, np. RFC 9110 to HTTP



Skąd się wziął REST?

1991

WWW i pierwszy HTTP

1996

HTTP/1.0 (RFC 1945)

1999

HTTP/1.1 (RFC 2616)

2000

Roy Fielding opisuje REST w doktoracie

2015

OpenAPI Initiative

2022

RFC 9110: obecna semantyka HTTP

REST nie jest wynalazkiem „na API”. To opis zasad, dzięki którym **sam WWW** dobrze się skaluje.



REST to styl, nie protokół

Podejście	Dane	Jak rozmawia	Gdzie spotkasz
REST	Zwykle JSON	Wiele adresów, metody i kody HTTP	Publiczne API, aplikacje web i mobilne
SOAP	XML w „kopercie”	Zwykle jeden adres i POST	Banki, systemy firmowe, starsze usługi
GraphQL	JSON, klient opisuje, czego chce	Jeden adres, język zapytań	Rozbudowane frontendy
gRPC	Protocol Buffers (binarnie)	HTTP/2, wywołania funkcji	Komunikacja między mikrousługami

REST nie ma „biblioteki REST” ani pliku standardu. To **zbiór zasad** nałożonych na HTTP.



Zasób i jego reprezentacje

ZASÓB

Uczeń nr 1

`/uczniowie/1`

Ma typ, dane, relacje (oceny) i operacje



```
Accept: …/json          {"id":1,"imie":"Ala"}
Accept: text/html       <li>Ala (4TI)</li>
Accept: …/xml           <uczen id="1">Ala</uczen>
```

Klient nigdy nie dostaje „ucznia z bazy”, tylko jego **reprezentację**. Stąd nazwa: **RE**presentational **S**tate **T**ransfer.



Sześć zasad architektury REST

01

Klient–serwer

Dwie niezależne strony

02

Bezstanowość

Serwer nie pamięta klienta

03

Buforowalność

Odpowiedzi można zapamiętać

04

System warstwowy

Po drodze mogą być pośrednicy

05

Jednolity interfejs

Te same reguły dla każdego zasobu

06

Kod na żądanie

Opcjonalna: serwer wysyła kod

by ***Mateusz Węglewski***

01 · Klient–serwer

KLIENT

Wygląd i obsługa użytkownika

Aplikacja mobilna, strona, inny program



API

HTTP + JSON

SERWER

Dane, logika, bezpieczeństwo

Express, baza danych, klucze

Każda strona zmienia się **niezależnie**, dopóki API zostaje takie samo.

02 · Bezstanowość

ZE STANEM (NIE REST)

```
1. POST /logowanie
   serwer zapamiętuje: sesja = Ala
2. GET /moje-oceny
   „kim jesteś?" → patrzy w pamięć
```

Drugi serwer tej sesji nie zna. Restart = wszyscy wylogowani

BEZSTANOWO (REST)

```
1. GET /uczniowie/1/oceny
   x-api-key: tajne123
2. GET /uczniowie/1
   x-api-key: tajne123
```

Każde żądanie ma wszystko. Odpowie dowolny serwer

Żądanie musi zawierać: **kim jestem**, **czego chcę** i **co z tym zrobić**.



03 · Buforowalność (cache)

Klient → **Cache** przeglądarka, proxy → **Serwer**

`Cache-Control: max-age=60`

Przez 60 s kolejne GET obsłuży cache. Serwer nawet ich nie zobaczy

`ETag → 304 Not Modified`

„Czy coś się zmieniło?" Nie? Serwer odsyła tylko kod, bez danych

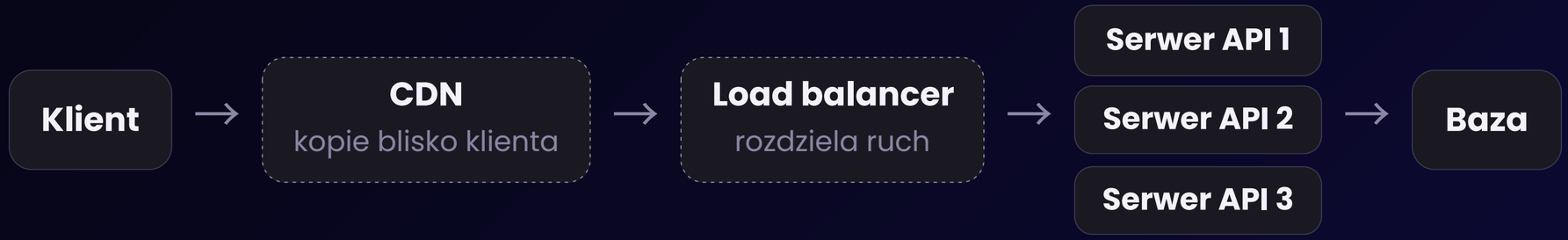
`Cache-Control: no-store`

Dane prywatne, np. oceny. Nigdzie nie zapisywać

Cache (pamięć podręczna) to szybki, tymczasowy magazyn danych, który przechowuje kopie często używanych informacji, aby procesor, aplikacja lub przeglądarka mogły do nich dotrzeć szybciej niż ze źródła głównego. Morele +2

by ***Mateusz Węglewski***

04 · System warstwowy



Klient **nie wie**, czy rozmawia z serwerem, czy z pośrednikiem. Widzi tylko adres API.

06 · Kod na żądanie

Serwer może wysłać kod do uruchomienia, np. JavaScript dla przeglądarki

by ***Mateusz Węglewski***

05 · Jednolity interfejs

Każdy zasób ma adres

`/uczniowie/1`

Zmiana przez reprezentację

`PUT /uczniowie/1 + JSON`

Komunikat opisuje sam siebie

`201 Created · Content-Type`

Linki do kolejnych kroków (HATEOAS)

`"oceny": "/uczniowie/1/oceny"`

Umiesz obsłużyć **/uczniowie**? To umiesz **/klasy** i **/przedmioty**.



Jak bardzo REST? Model Richardsona

Poziom 0

Jeden adres, wszystko POST

`POST /api`

Poziom 1

Zasoby

`POST /uczniowie/1`

Poziom 2

Metody i kody HTTP

`GET /uczniowie/1 → 200`

← tu jesteśmy

Poziom 3

Hipermedia

Odpowiedź zawiera linki: co można zrobić dalej

Źródło: M. Fowler, „Richardson Maturity Model", 2010

by ***Mateusz Węglewski***

Metody HTTP i CRUD

Metoda	CRUD	Co robi	Sukces	Bezpieczna	Idempotentna
GET	Read	Pobiera zasób lub listę	200	tak	tak
POST	Create	Tworzy zasób w kolekcji	201 + Location	nie	nie
PUT	Update	Zastępuje cały zasób	200	nie	tak
PATCH	Update	Zmienia część zasobu	200	nie	nie zawsze
DELETE	Delete	Usuwa zasób	204	nie	tak

CRUD = Create, Read, Update, Delete. **Bezpieczna** = niczego nie zmienia.
Idempotentna = powtórka daje ten sam stan serwera.

Źródło: RFC 9110, rozdz. 9 · MDN: HTTP request methods

by ***Mateusz Węglewski***

PUT czy PATCH?

PUT: oto cały nowy uczeń

```
PUT /uczniowie/2
{ "imie": "Olek",
  "nazwisko": "Nowak",
  "klasa": "5TI" }
```

Brakuje pola? **400**. Zasób jest dokładnie taki, jak w body

PATCH: zmień tylko to

```
PATCH /uczniowie/2
{ "klasa": "5TI" }
```

Imię i nazwisko **zostają**. Zmienia się tylko klasa



IDEMPOTENTNOŚĆ

Raz czy dziesięć razy: skutek ten sam.

Jak przycisk windy: kolejne naciśnięcia nie wezwą drugiej windy.

L7

by ***Mateusz Węglewski***

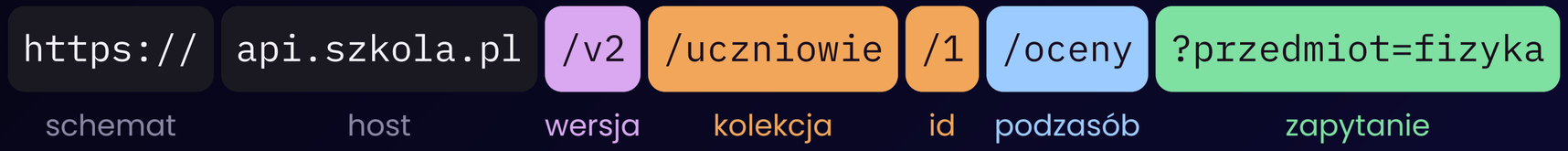
Po co to wszystko? Sieć zawodzi

1 Klient wysyła **POST** z Kasią → **2** Serwer zapisuje ją jako **id 6** → **3** Odpowiedź **ginie** w słabym Wi-Fi → **4** Klient ponawia: **id 7**, duplikat

GET, PUT i DELETE można **bezpiecznie ponowić**.
POST **nie**, chyba że dodamy ochronę.



Anatomia adresu w REST



ŚCIEŻKA = CO

Który zasób: oceny ucznia nr 1

ZAPYTANIE = JAK

Filtr, sortowanie, strona wyników

METODA = CO ZROBIĆ

Nie ma jej w adresie

Jak nazywać endpointy

Źle	Dobrze	Zasada
GET /pobierzUczniow	GET /uczniowie	Rzeczownik, nie czasownik
POST /uczniowie/5/usun	DELETE /uczniowie/5	Akcję mówi metoda
/uczen/5 obok /uczniowie	/uczniowie/5	Zawsze liczba mnoga
/Lista_Uczniow	/uczniowie	Małe litery, myślnik
/oceny?uczen=1	/uczniowie/1/oceny	Relacja w ścieżce
/uczniowie.json	/uczniowie	Format w Content-Type



Filtruj, sortuj, stronicuj

`/uczniowie?` kolekcja

`klasa=4TI` filtr

`&sort=-nazwisko` minus = malejąco

`&strona=1&naStrone=2` stronicowanie

```
{
  "dane": [ { "id": 3, "imie": "Basia", … }, { "id": 4, … } ],
  "meta": { "strona": 1, "naStrone": 2, "razem": 3 }
}
```




Jakie kody zwraca dobre API

Metoda	Sukces	Typowe błędy klienta
GET	200	404 nie ma zasobu
POST	201 + Location	400 złe dane · 409 już istnieje
PUT, PATCH	200	400 złe dane · 404 nie ma zasobu
DELETE	204	404 nie ma zasobu
Każda	—	401 kim jesteś? · 403 nie wolno · 405 zła metoda

Błąd zawsze w tym samym kształcie: {"blad": "…", "id": "…"} (L5)

by ***Mateusz Węglewski***

Która zmiana psuje klientów?

Zmiana w API	Dla starych klientów
Nowe pole w odpowiedzi	bezpieczna
Nowy endpoint	bezpieczna
Nowe opcjonalne pole w body	bezpieczna
Zmiana nazwy lub usunięcie pola	łamiąca
Zmiana typu pola: "5" na 5	łamiąca
Nowe wymagane pole w body	łamiąca
Inny kod statusu dla tej samej sytuacji	łamiąca

Zmiana łamiąca (breaking change): stary klient, który nic o niej nie wie, przestaje działać.



Gdzie umieścić wersję?

ŚCIEŻKA · NASZ WYBÓR

`/api/v2/uczniowie`

Widać w logach i w przeglądarce. Najczęściej spotykane

NAGŁÓWEK

`Accept: …v2+json`

Czyste adresy, ale trudniej testować w przeglądarce

QUERY

`/uczniowie?wersja=2`

Łatwo zapomnieć, miesza się z filtrami

Stara wersja działa dalej i ogłasza koniec: **Deprecation**, **Sunset**, **Link** do następczyni



Umowa na piśmie: OpenAPI

openapi.yaml (fragment)

```yaml
openapi: 3.1.0
info:
  title: API szkoły
  version: 2.0.0
paths:
  /uczniowie/{id}:
    get:
      summary: Pobierz ucznia
      responses:
        '200': { description: Uczeń }
        '404': { description: Nie ma ucznia }
```

Opis API, który czyta **człowiek i program**

Z pliku powstaje **dokumentacja**, np. Swagger UI

...a także **kod klientów** i testy

Standard **OpenAPI Initiative**, Linux Foundation



Słowniczek: dokumentacja API

OpenAPI
Standard: jak zapisać opis REST API w pliku YAML lub JSON

Swagger
Narzędzia do OpenAPI, np. Swagger UI. Dawna nazwa standardu

YAML
Format pliku, w którym strukturę wyznaczają wcięcia

Schemat (schema)
Opis kształtu danych: jakie pola, jakie typy, które wymagane

$ref
Odwołanie do schematu zapisanego raz w components

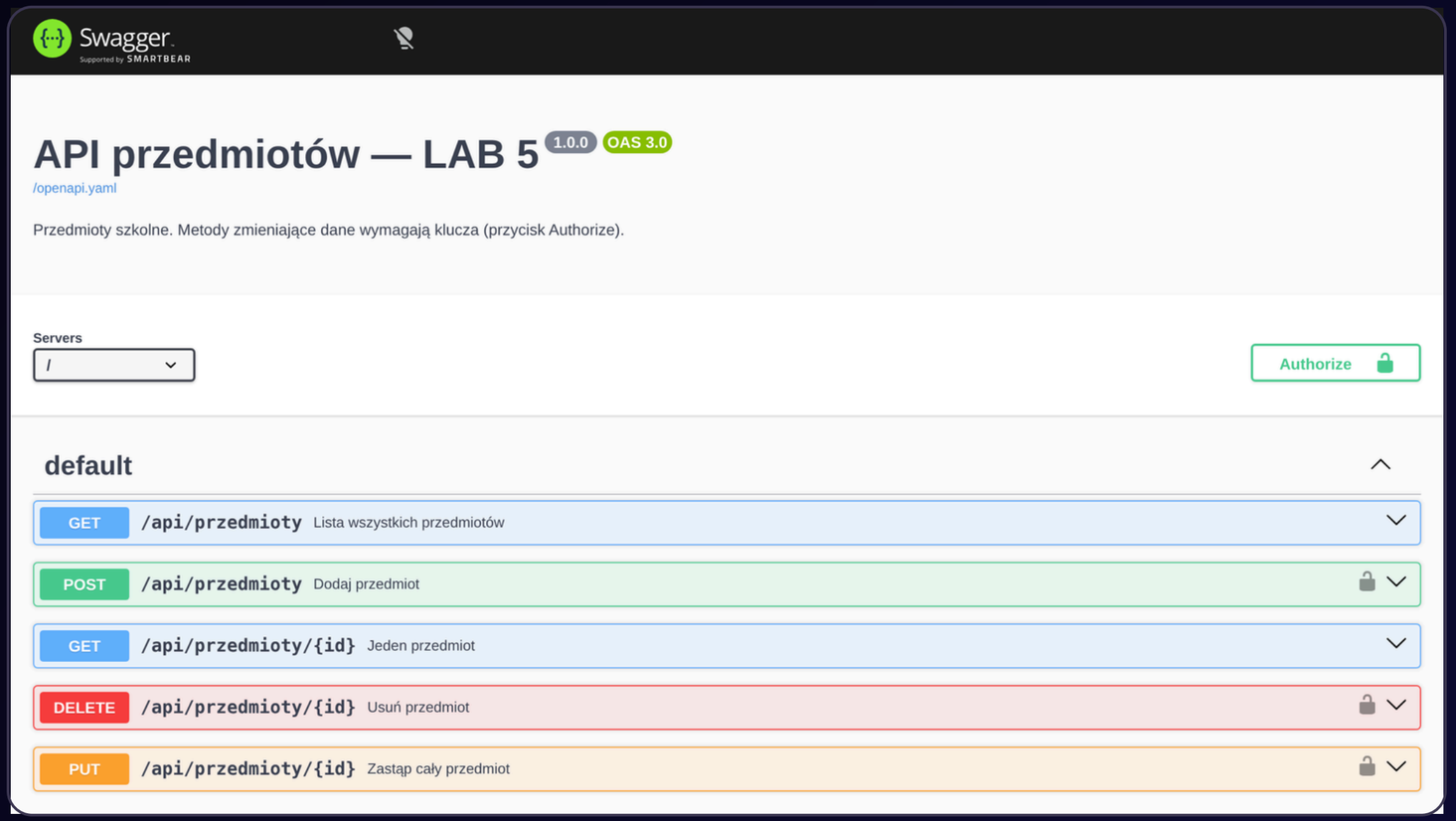
securitySchemes
Opis uwierzytelniania, np. klucz API w nagłówku

Postman / Insomnia
Programy do wysyłania żądań HTTP w okienkach zamiast curl

Kolekcja
Zapisany zestaw zapytań do jednego API w Postmanie



Swagger UI: dokumentacja do klikania



Kolor = metoda: GET niebieski, POST zielony, PUT pomarańczowy, DELETE czerwony

Kłódka: trasa wymaga klucza

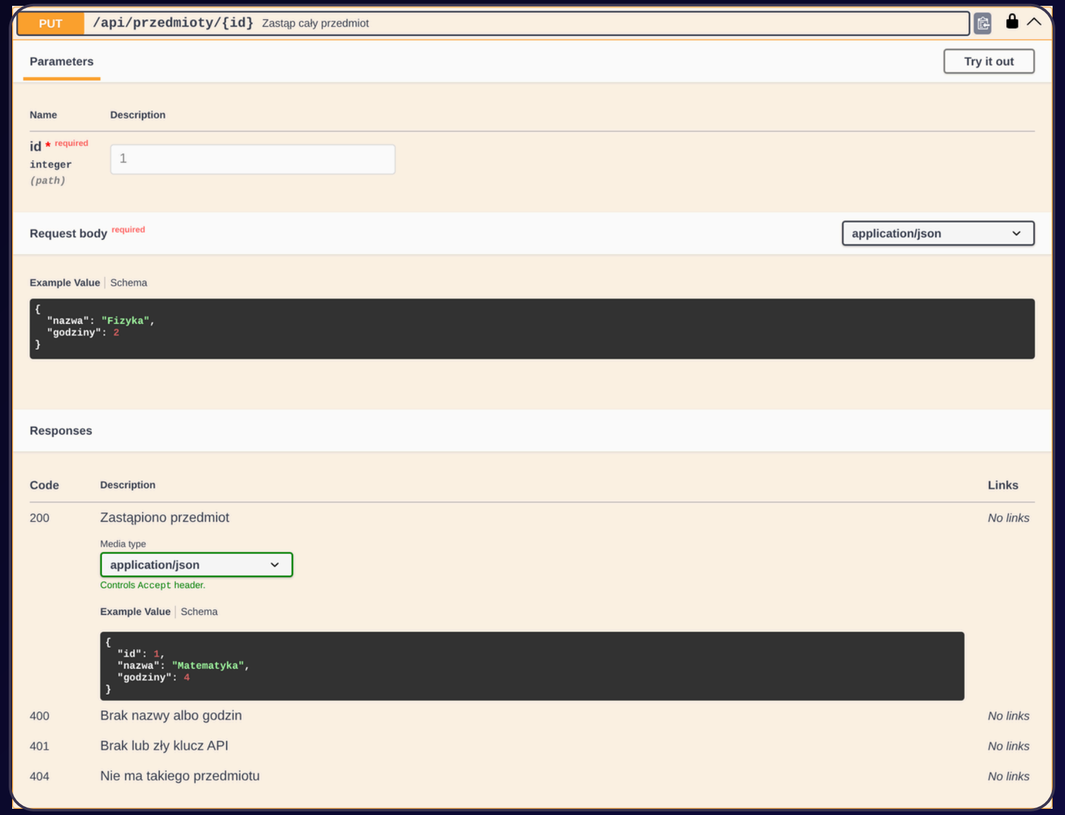
Cała strona powstała z jednego pliku openapi.yaml

by ***Mateusz Węglewski***

Z pliku YAML na stronę

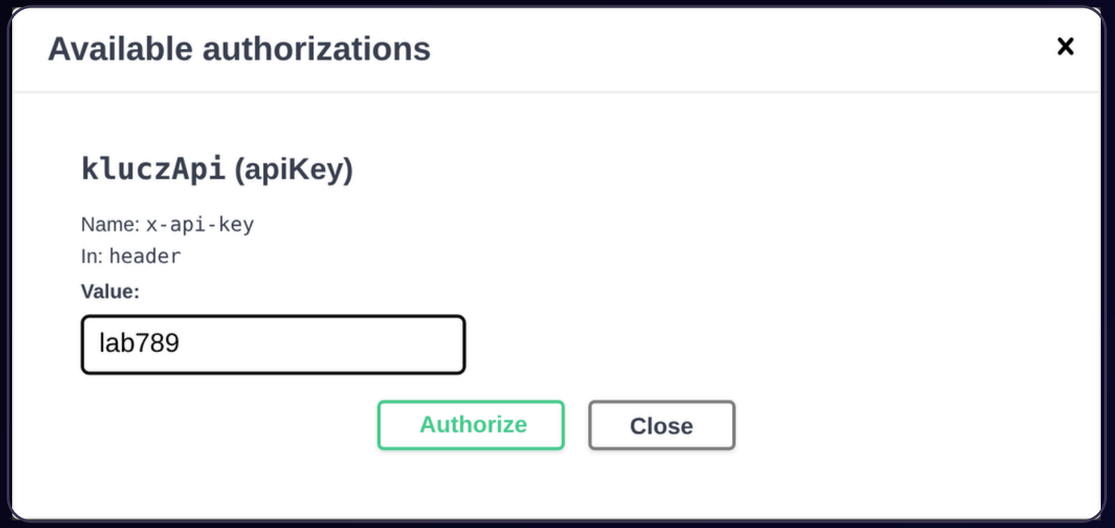
openapi.yaml

```yaml
put:
  summary: Zastąp cały przedmiot
  security:
    - kluczApi: []
  requestBody:
    … NowyPrzedmiot
  responses:
    '200': Zastąpiono przedmiot
    '400': Brak nazwy albo godzin
    '401': Brak lub zły klucz API
    '404': Nie ma takiego przedmiotu
```



by ***Mateusz Węglewski***

Testowanie: Authorize i Try it out



1. **Authorize**: wpisz klucz raz
2. **Try it out** przy trasie
3. Uzupełnij id i body, **Execute**
4. Czytaj: **curl**, kod, body, nagłówki

Błąd oczami klienta

Curl

```
curl -X 'PUT' \
  'http://localhost:8080/api/przedmioty/2' \
  -H 'accept: */*' \
  -H 'x-api-key: lab789' \
  -H 'Content-Type: application/json' \
  -d '{
  "nazwa": "Fizyka"
}'
```

Request URL

```
http://localhost:8080/api/przedmioty/2
```

Server response

Code	Details
400	Error: Bad Request

Response body

```
{
  "blad": "Wymagane pola: nazwa, godziny"
}
```

Download

Response headers

```
connection: keep-alive
content-length: 40
content-type: application/json; charset=utf-8
date: Sat,07 Nov 2026 10:42:40 GMT
etag: W/"28-qP7woW6xOSCm+6v7L5KgAMCkEvc"
keep-alive: timeout=5
x-powered-by: Express
```

Responses

400: kod mówi, czyja wina

blad: mówi, co poprawić

content-type: odpowiedź to JSON, także przy błędzie

by ***Mateusz Węglewski***

Standardowy format błędu

NASZ, PROSTY

```
HTTP/1.1 404 Not Found
Content-Type: application/json

{
  "blad": "Nie ma takiego przedmiotu",
  "id": "9d20577d"
}
```

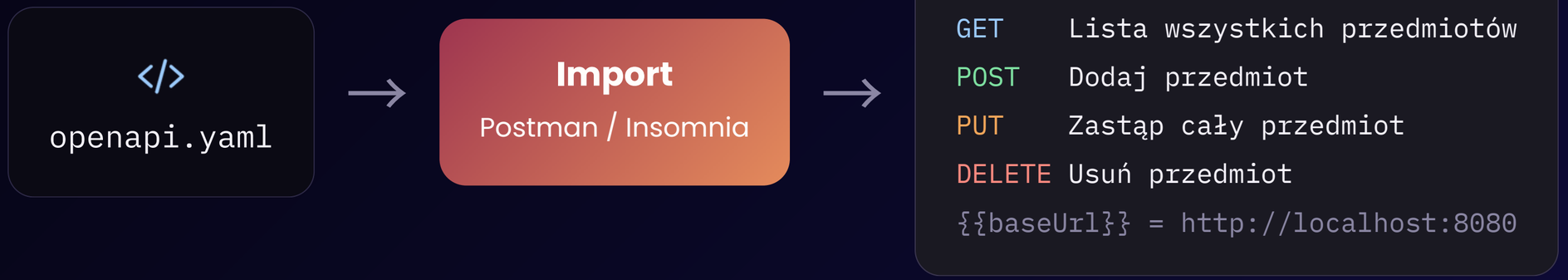
RFC 9457: PROBLEM DETAILS

```
HTTP/1.1 404 Not Found
Content-Type: application/problem+json

{
  "type": "https://api.szkola.pl/bledy/brak",
  "title": "Nie ma takiego przedmiotu",
  "status": 404,
  "detail": "Przedmiot o id 99 nie istnieje",
  "instance": "/api/przedmioty/99"
}
```




Postman i Insomnia: import OpenAPI



Jeden opis API: **dokumentacja** w Swaggerze i **gotowe zapytania** w Postmanie.

LIVE DEMO

Teoria w nagłówkach

```
$ A=localhost:3000/api/v2/uczniowie
$ curl -i $A/1                                  # Content-Type, ETag
$ curl -i -X DELETE $A -H "x-api-key: tajne123"  # 405 + Allow
$ curl -i localhost:3000/api/v1/uczniowie/1          # Deprecation
```

Każda zasada z wykładu jest widoczna w **surowej odpowiedzi HTTP**.





APLIKACJE WEBOWE

:wq

L7



by ***Mateusz Węglewski***